ToolCrib Licensing Architecture & Cryptographic Specification

Comprehensive Technical Guide to Key Generation, Encryption/Encoding, Decoding, and Multi-Layered Validation

System Overview: ToolCrib Software utilizes an enterprise-grade, asymmetric cryptographic licensing system powered by **Ed25519 (Edwards-curve Digital Signature Algorithm)**, **SHA-256 cryptographic hashing**, and **URL-Safe Base64 serialization**. This architecture ensures complete offline operation, hardware node-locking via MAC address verification, payload integrity verification, and non-forgeable signature authentication.

1. Core Cryptographic Algorithms & Standards

The ToolCrib licensing module (license_utils.py) integrates multiple industry-standard algorithms to achieve security, offline autonomy, and tamper prevention:

Algorithm / Primitive	Standard / Spec	Role in Licensing Flow
Ed25519 (Edwards-curve DSA)	RFC 8037 / Curve25519	Asymmetric digital signature. The vendor signs payload bytes with a 32-byte private key. The server validates using the matching public key.
SHA-256 (Secure Hash Standard)	FIPS PUB 180-4	Computes a 256-bit cryptographic digest over canonical base metadata claims to guarantee complete payload integrity.
URL-Safe Base64 (Base64URL)	RFC 4648 § 5	Encodes binary signatures and JSON byte payloads into clean ASCII strings without padding ('=') for easy transport.
Canonical JSON (RFC 8785)	ECMAScript / RFC 8785	Ensures deterministic key sorting (sort_keys=True) and compact whitespace elimination before hashing & signing.

2. License Key Generation Flow & Architecture

License generation occurs on the vendor side via generate_license_key() or the CLI tool generate_license.py. The flow follows a strict 7-step sequence:

- **Step 1 — Input Collection & MAC Normalization:** Accept target MAC address, customer company name, and expiration days. The MAC address is normalized to uppercase XX:XX:XX:XX:XX:XX format.
- **Step 2 — Base Claims Construction:** Construct JSON claims containing customer, mac_address, ISO-8601 UTC timestamps (issued_at, expires_at), and license_version: '2.0'.

- **Step 3 — SHA-256 Checksum Computation:** Convert base claims into canonical UTF-8 JSON bytes (sorted keys, no spaces) and derive SHA-256 checksum hex string.
- **Step 4 — Full Payload Assembly & Canonicalization:** Embed sha256_checksum into payload object and re-canonicalize into UTF-8 binary bytes.
- **Step 5 — Ed25519 Asymmetric Digital Signing:** Sign payload bytes using the vendor's private key (Ed25519PrivateKey.sign(payload_bytes)) to produce a 64-byte raw signature.
- **Step 6 — Document Packaging & Base64URL Encoding:** Encode payload bytes and signature bytes into URL-safe Base64 strings inside a document dictionary: {'payload': ..., 'signature': ...}.
- **Step 7 — Final Key Serialization:** Convert document dictionary to canonical JSON bytes and perform outer Base64URL encoding to produce the final license string.

Payload & Signed Document Schema Structure:

```
// Internal Unpacked Payload Structure (JSON) { "customer": "ACME Industrial Corp", "mac_address": "00:1A:2B:3C:4D:5E", "issued_at": "2026-09-24T10:00:00.000000", "expires_at": "2027-09-24T10:00:00.000000", "license_version": "2.0", "sha256_checksum": "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855" } // Outer Signed License Container (Base64URL Enclosed) { "payload": "eyJjdXN0b21lciI6ICJBQ01FIEluZHVzdHJpYWwgQ29ycCIsIC4uLn0", "signature": "V3ZpODNqZjI5M2ZqczI5M2pmczI5M2pmczI5M2pmczI5M2pmczI5M2pm..." }
```

3. License Key Decoding ('Decrypting') & Unpacking

In asymmetric signing schemes, public key licensing does not use symmetric encryption (like AES); instead, it uses **Asymmetric Digital Signature Packaging with URL-Safe Base64 Decoding**. The term 'decryption' in this architecture refers to reversing the Base64URL encoding layers and extracting verifiable claims:

- **Outer Base64URL Decoding:** The license key string is stripped of whitespace and passed to _decode_b64url(), restoring URL padding ('=') and decoding ASCII bytes into UTF-8 JSON document string.
- **Document Parsing:** JSON parser extracts payload (Base64URL encoded string) and signature (Base64URL encoded signature string).
- **Payload & Signature Extraction:** _decode_b64url(payload) yields raw UTF-8 canonical JSON payload bytes. _decode_b64url(signature) yields 64 raw binary signature bytes.
- **Claims Deserialization:** JSON parser deserializes payload bytes into Python dictionary containing customer name, target MAC address, validity timestamps, and SHA-256 hash.

4. Multi-Tiered License Validation Pipeline

When verify_license_key() is invoked by backend API endpoints (/api/license/status, /api/license/validate, /api/license/activate), the key must successfully pass through **5 consecutive security validation gates**:

Validation Gate	Mechanism & Function	Failure Cause & Security Defense

Gate 1: Ed25519 Signature Check	Executes public_key.verify(signature_bytes, payload_bytes) using vendor public key (ed25519_public.b64).	Fails if key was modified or generated without vendor private key. Rejects tampered keys immediately.
Gate 2: SHA-256 Hash Integrity	Recomputes SHA-256 digest on base claims and compares with payload['sha256_checksum'].	Fails if internal metadata values (e.g. expiration date or customer name) were altered. Prevents internal claim tampering.
Gate 3: MAC Format Standard	Evaluates MAC regex pattern ^([0-9A-F]{2}:){5}[0-9A-F]{2}$ via is_valid_mac_pattern().	Fails if MAC address field is malformed or corrupted.
Gate 4: Hardware Binding Match	Queries all local host Network Interfaces (via psutil & uuid) and matches license MAC.	Fails if license key is copied to unauthorized hardware. Enforces host node-locking.
Gate 5: Expiration Timestamp	Compares system UTC clock (datetime.utcnow()) against ISO-8601 expires_at field.	Fails if current date exceeds expiration timestamp (unless perpetual license where expires_at=None).

5. End-to-End License Lifecycle Workflow

The complete operational lifecycle spanning Vendor Generation, Client Server Activation, and API Enforcement is summarized below:

[VENDOR ENVIRONMENT]
1. Vendor runs: python generate_license.py --mac "XX:XX:XX..." --company "Acme" --days 365
2. System loads ed25519_private.b64 & builds canonical JSON payload + SHA-256 hash.
3. Ed25519 signature generated -> Wrapped in JSON document -> Outer Base64URL encoded string output.

[CLIENT APP SERVER / ACTIVATION API]
4. Admin pastes key into ToolCrib UI -> Sent to POST /api/license/activate.
5. Server decodes Base64URL outer layer & extracts payload + signature.
6. Public Key ed25519_public.b64 verifies Ed25519 signature.
7. System recomputes SHA-256 hash, validates MAC address match against host NICs, and checks expiry date.

[DATABASE & SYSTEM ENFORCEMENT]
8. Validated key stored in toolcrib.license_info (sets is_active='yes'). Company profile registered in toolcrib.companies.
9. GET /api/license/status checks active license on app startup & routine API access.

6. Backend Source Code Reference

File Path	Primary Function / Endpoint	Responsibility
backend/license_utils.py	generate_license_key() verify_license_key() get_or_create_keypair()	Core cryptographic library. Handles Ed25519 keypair loading, Base64URL encoding/decoding, SHA-256 hashing, signing & 5-stage verification.
backend/generate_license.py	CLI Interface (main())	Standalone vendor CLI script to generate signed license keys for customers given MAC address and duration.
backend/license_routes.py	/api/license/status /api/license/validate /api/license/activate	FastAPI endpoints for checking software status, validating draft keys, and activating licenses into database.
backend/models.py	LicenseInfo ORM Model	SQLAlchemy database schema storing active/historical license keys, MACs, signatures, hashes, and validity dates.

Security Verification Guarantee: Ed25519 signature verification mathematically prevents key forgery. SHA-256 hashing prevents field modification. Host MAC binding prevents unauthorized hardware transfer. Expiration dates enforce subscription compliance. The system is 100% offline-capable and requires no external licensing server call.